

Mikhail Razakov

Compiler Backend / Code Generation Engineer

razakov.mikhail@outlook.com
github.com/Misha1302
LinkedIn

Compiler engineer with professional LLVM 22 experience at MCST and a separate x86-64 backend project. I implemented the path from SSA/CFG through liveness/interference, register allocation/spills, and phi lowering to SysV x86-64; generated code runs in a separate process and is compared against a reference interpreter.

EXPERIENCE

- 2026 — present** **ISP RAS — Static Analysis Engineer**
- Contribute to SharpChecker, ISP RAS's C#.NET static-analysis platform.
- July — August 2026** **MCST — Compiler Engineering Intern**
- Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.
 - Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.

PROJECTS

x86-64 Compiler Backend & Register Allocation

SSA/CFG validation → liveness/interference → register allocation/spills → phi lowering → SysV x86-64 emission.

Assignment verifier plus native-vs-interpreter differential execution; the current project CI is green.

LLVM Interprocedural Global IV Optimization

An LLVM 22 pass localizes supported induction-like globals after interprocedural effect analysis, affine evolution, and a separate applicability check; APInt models fixed-width wraparound.

29 positive/negative regression cases cover LLVM Verifier checks, structural expectations, idempotence, and observable before/after behavior.

UniversalToolchain

Dependency/conflict resolution, implementation selection, pass ordering, and typed artifact routes before execution.

1,324/1,324 tests; interpreter/CIL parity and configuration checks before runtime.

SKILLS

Backend / Codegen

SysV x86-64, liveness/interference, register allocation, spills, phi lowering, native execution

LLVM / IR

C++23, LLVM 22/IR, CFG/SSA, loop/interprocedural analysis, IR transformations

Testing

assignment verifier, LLVM Verifier, structural checks, idempotence, differential execution

Languages

C++23, Rust, C#, .NET, Linux, CMake

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

LangDev'26 — Build the Language, Then Make the Abstractions Disappear

Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.

UniversalToolchain — Baltic Science and Engineering Competition

First Degree Diploma and Grand Prize "Perfection as Hope" for UniversalToolchain.

MEPhI Junior

Overall winner; First Degree Diploma, 96/100 for UniversalToolchain.

Moscow / remote